

Universidad de Nueva Extremadura

Facultad de Ingeniería

Planificación

Material No. 4

Ingeniería de Software II

Ing. Mauricio Moo Aguilar

Modelo de Arquitectura Basado en Capas

La arquitectura de software es un componente esencial en el desarrollo de sistemas informáticos, ya que define la forma en que sus diferentes partes se organizan e interactúan. Uno de los modelos más utilizados por su claridad, escalabilidad y mantenimiento sencillo es el modelo de arquitectura basado en capas. Este modelo organiza el sistema en niveles jerárquicos, donde cada capa tiene una función específica y se comunica con las capas adyacentes.

La arquitectura en capas es un patrón estructural donde el sistema se divide en distintos niveles, conocidos como "capas", cada una con responsabilidades claras. Cada capa depende solamente de la capa inferior inmediata y ofrece servicios a la capa superior.

Este modelo promueve la separación de responsabilidades, facilitando la comprensión del sistema, su mantenimiento, escalabilidad y pruebas. Cada capa puede ser modificada sin afectar las demás, siempre que se mantengan las interfaces.

Ventajas del modelo en capas

- **Separación de responsabilidades**: Cada capa tiene una función específica, lo que hace más claro el diseño del sistema.
- **Facilidad de mantenimiento**: Si se requiere hacer un cambio, puede hacerse en una sola capa sin afectar las otras.
- **Reutilización**: Las capas inferiores, como la de acceso a datos, pueden ser utilizadas por varios sistemas diferentes.
- **Escalabilidad**: Es fácil agregar nuevas funcionalidades sin romper la estructura existente.
- **Pruebas más simples**: Se puede probar cada capa de forma aislada.

Principales capas del modelo

Aunque puede haber más capas dependiendo del sistema, las más comunes son:

1. Capa de Presentación (UI o Interfaz de Usuario)

Es la capa con la que el usuario interactúa directamente. Su objetivo principal es mostrar la información y capturar entradas del usuario.

Responsabilidades:

- Mostrar datos de manera clara.
- Validar la entrada básica del usuario.
- Enviar las solicitudes al sistema.

Ejemplo: Formularios web, menús de navegación, botones, aplicaciones móviles.

2. Capa de Lógica de Negocio (o Capa de Aplicación)

Contiene la lógica del sistema, es decir, cómo se procesan los datos, las reglas del negocio y el flujo de trabajo.

Responsabilidades:

- Procesar las solicitudes del usuario.
- Aplicar las reglas de negocio.
- Coordinar la comunicación entre las capas de presentación y de datos.

Ejemplo: Cálculo de descuentos, validación de pedidos, reglas de seguridad.

3. Capa de Acceso a Datos

Es la capa encargada de comunicarse con las bases de datos u otras fuentes de datos. Sirve como intermediario entre la lógica del negocio y el almacenamiento.

Responsabilidades:

- Leer y escribir datos desde y hacia las fuentes de almacenamiento.
- Traducir objetos del sistema a estructuras que puedan almacenarse.

Ejemplo: Consultas SQL, acceso a archivos, manejo de conexiones con bases de datos.

4. Capa de Datos (Persistencia)

Aquí es donde residen físicamente los datos. Aunque en algunos modelos esta capa se considera parte del entorno, es común incluirla para representar el almacenamiento real.

Responsabilidades:

- Guardar permanentemente la información.
- Asegurar la integridad y seguridad de los datos.

Ejemplo: Bases de datos relacionales (MySQL, PostgreSQL), archivos planos, servicios en la nube.

Ejemplo Práctico

Imaginemos una aplicación de biblioteca universitaria donde los alumnos pueden consultar la disponibilidad de libros y hacer préstamos en línea.

- **Capa de Presentación**: Una página web donde el usuario puede buscar libros, iniciar sesión y realizar solicitudes.
- **Capa de Lógica de Negocio**: Procesa la solicitud de préstamo, valida que el usuario esté registrado y que el libro esté disponible.
- **Capa de Acceso a Datos**: Ejecuta la consulta en la base de datos para verificar si el libro está disponible.
- **Capa de Datos**: Contiene tablas de libros, usuarios y préstamos.

Este ejemplo muestra cómo las capas se comunican entre sí sin romper la independencia de cada una.

Buenas prácticas al implementar arquitectura en capas

1. **No saltarse capas**: Cada capa debe comunicarse solo con la inmediatamente inferior o superior.
2. **Mantener interfaces claras**: Las funciones públicas de cada capa deben ser bien definidas y estables.
3. **Evitar lógica duplicada**: Toda lógica de negocio debe estar centralizada en su capa correspondiente.
4. **Inyectar dependencias**: Usar patrones como Inversión de Control (IoC) o inyección de dependencias para desacoplar componentes.

Limitaciones del modelo

Aunque es ampliamente utilizado, este modelo también presenta algunos desafíos:

- **Puede volverse rígido**: Si no se diseña correctamente, la estructura puede dificultar cambios complejos.
- **Sobreingeniería** en proyectos pequeños: Para aplicaciones simples, el uso de múltiples capas puede ser innecesario.
- **Desempeño**: Las múltiples llamadas entre capas pueden impactar el rendimiento si no se optimizan.

Conclusiones

El modelo de arquitectura en capas es una estrategia poderosa para organizar el diseño de sistemas de software de manera estructurada, clara y mantenible. Para estudiantes de Ingeniería en Sistemas, comprender este modelo no solo es útil para desarrollar aplicaciones escalables y limpias, sino también para aprender a aplicar principios de diseño que son fundamentales en la industria del software.

Adoptar este patrón ayuda a construir sistemas robustos, reutilizables y listos para crecer según las necesidades del negocio, facilitando además el trabajo en equipo y la implementación de pruebas unitarias y de integración.

Fuentes Bibliográficas:

Oscar Javier Blancarte Iturralde – (2020) Introducción a la arquitectura de software: Un enfoque práctico, Publicación Independiente.